Operating System and System Programming (25CS2104E)

Multi-Process Task Manager for Linux

OS Hackathon

Problem Statement: Develop a lightweight task manager that displays process information and allows users to control selected processes.

Under the Esteemed Guidance of

Dr. M. Raghupati

CSE Department

K L (Deemed to be) University

Team Members:

B. Deepak Goud – 2520080035

A. Lakshmi Sai Teja – 2520080005

E. Pranay – 2520080061

Contents

- Problem Statement
- Objectives and Features
- System Architecture
- Technologies Used
- Implementation Approach
- Key Calculations
- GUI Design and Workflow
- Operating Systems Concepts
- Testing
- Results and Demonstration
- Impact and Future Scope
- Conclusion and Q&A

Problem Statement

Problem Statement

Develop a lightweight task manager that displays process information and allows users to control selected processes.

Challenges of monitoring and controlling Linux processes

1

Hidden process data

Process details are spread across many text files under /proc.

2

Usage must be calculated

CPU % needs two samples over time; memory values need conversion.

3

Risky process control

Signalling the wrong PID can stop important programs; permissions apply.

4

Command-line only tools

ps, top and kill need commands; beginners prefer a simple GUI.

Our approach: a lightweight C + GTK 3 task manager that reads /proc and controls processes with signals

Objectives and Features

Objective

Build a lightweight Linux task manager that shows live process information and lets users control selected processes.

1 Process Listing

Shows running processes with PID, name and other details

2 Search

Quickly finds a process in the list

3 CPU Monitoring

Shows the CPU usage of each process

4 Memory Monitoring

Shows the memory usage of each process

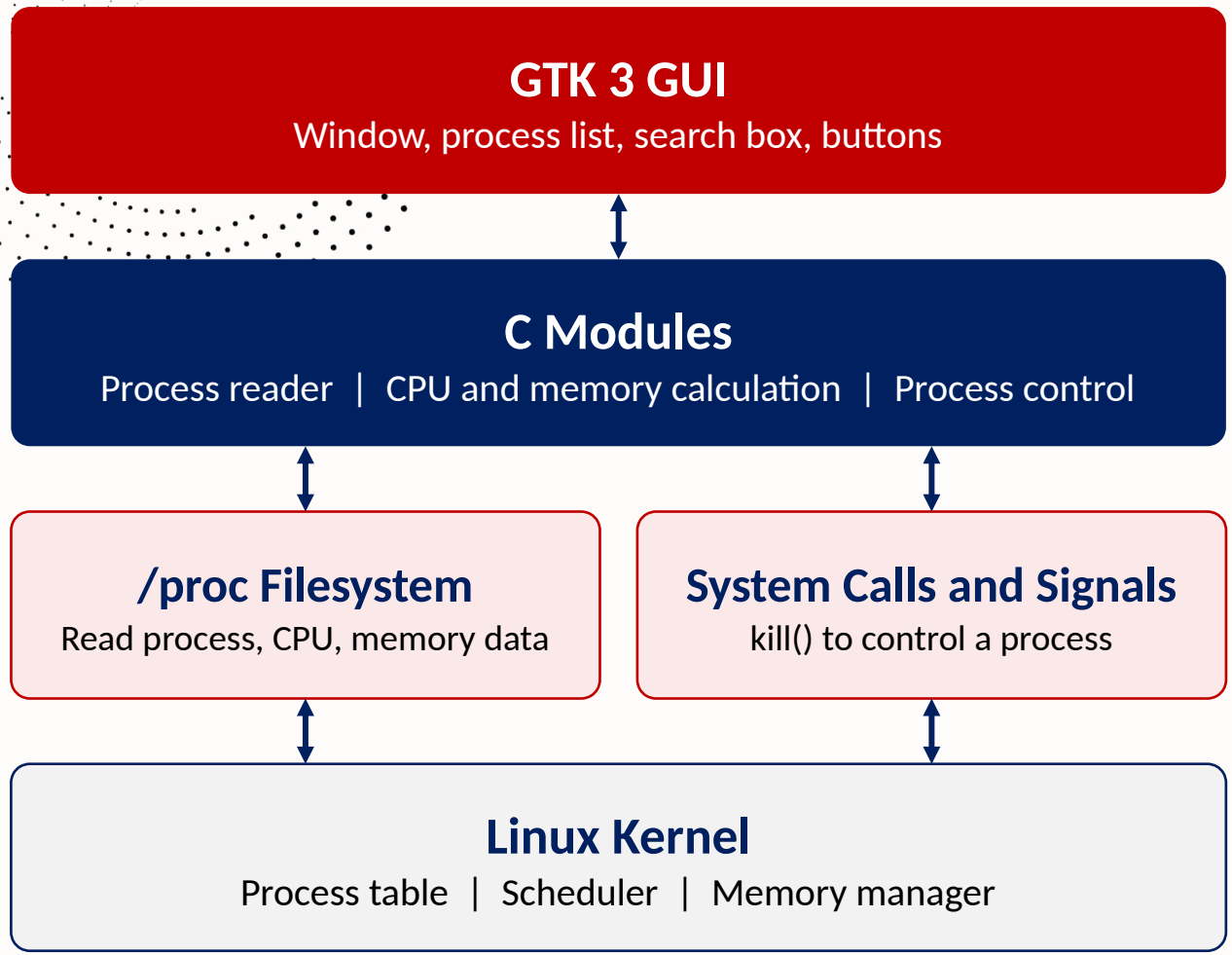
5 Automatic Refresh

Updates the information at regular intervals

6 Process Control

Select a process and send it a signal to terminate it

System Architecture



Read path (monitoring)

- Kernel exposes data in /proc files
- C modules read and parse the files
- CPU % and memory % are calculated
- GTK 3 GUI shows the results

Control path (termination)

- User selects a process in the GUI
- C module calls kill(pid, signal)
- Kernel checks permission, delivers signal

Technologies Used

Technology	Purpose in the project
C language	Core logic and system-level programming
GTK 3	Graphical user interface (window, list view, buttons)
GCC	Compiles the C source code
Ubuntu Linux	Development and runtime platform
/proc filesystem	Source of process, CPU and memory information
Makefile	Automates building the project
Signals and system calls	kill() delivers signals for process control

Implementation Approach

1 Process Discovery

- Scan the /proc directory
- Folders with numeric names are PIDs

2 Information Parsing

- Read /proc/[pid]/stat
- Extract name, state and PPID

3 CPU Calculation

- Process time = utime + stime
- Compare two refreshes with total CPU time

4 Memory Calculation

- Read resident memory (RSS)
- Compare with MemTotal in /proc/meminfo

5 Process Selection

- User selects a row in the list
- Its PID is used for control

6 Termination

- Send signal: kill(pid, signal)
- Handle errors: EPERM, ESRCH

Implementation: Key Calculations

CPU usage (per process)

```
proc_time = utime + stime;     /* fields 14, 15 of /proc/[pid]/stat */
cpu_pct = (proc_time - prev_proc_time) * 100.0
        / (total_time - prev_total_time);
/* total_time = sum of the "cpu" line in /proc/stat */
```

Memory usage (per process)

```
rss_kb  = VmRSS from /proc/[pid]/status;   /* resident memory, kB */
mem_pct = rss_kb * 100.0 / mem_total_kb;   /* MemTotal: /proc/meminfo */
```

Process control (signal)

```
if (kill(pid, SIGTERM) == -1) {
    /* errno EPERM: no permission,  errno ESRCH: no such process */
}
```

Illustrative snippets of the Linux interfaces used; not copied from the project source code.

GUI Design and Workflow

Multi-Process Task Manager

Search process...

Terminate

PID	Process Name	CPU %	Memory %

Process list refreshes automatically; selected row is the target of Terminate.

Schematic of the GUI layout (not a screenshot)

1. **Launch**
   Application opens, process list loads
2. **Search**
   Type to find a process
3. **Monitor**
   Watch CPU and memory; list auto-refreshes
4. **Select**
   Click a process row
5. **Control**
   Press Terminate; signal is sent

Operating Systems Concepts

1 PID and PPID

PID identifies a process; PPID is its parent. Together they form the process tree.

2 Process States

R running, S sleeping, D disk wait, Z zombie, T stopped.

3 CPU Accounting

Kernel counts CPU time in clock ticks: utime (user) and stime (kernel).

4 Memory Usage

RSS is the part of a process held in RAM; compared with total RAM.

5 Signals

Messages sent to a process: SIGTERM asks it to exit, SIGKILL forces it.

6 Permissions

Users can signal only their own processes; root can signal any (else EPERM).

Testing

#	Test case	Expected result	Actual result
1	Launch the application	Window opens and process list is shown	*To be recorded*
2	Search for a process	Only matching processes are shown	*To be recorded*
3	Observe CPU and memory	Values shown and change on refresh	*To be recorded*
4	Wait for auto refresh	List updates without user action	*To be recorded*
5	Terminate own test process	Process disappears from the list	*To be recorded*
6	Terminate without permission	Error is reported; app keeps running	*To be recorded*
7	Terminate an exited PID	Handled safely; app keeps running	*To be recorded*

Error handling to check: no permission (EPERM), process already exited (ESRCH), and /proc files that disappear while reading

Results and Demonstration

Application screenshot

Insert from the live run: process list with CPU and memory columns

Live demo checklist

1. Build with make and launch the app
2. Show the process list and search
3. Watch CPU and memory update on refresh
4. Select a test process and terminate it
5. Show an error case (no permission / exited PID)

Impact and Future Scope

Impact

- Educational: shows OS concepts working on a real system
- Usability: simple GUI instead of terminal commands
- Lightweight: small C program with few dependencies

Limitations

- Linux only, because it depends on /proc
- Basic control: selected processes and signals
- Controlling other users' processes needs permission

Future Scope

- Sort and filter by any column
- CPU and memory history graphs
- Process tree view using PPID
- Change priority (nice) and send more signals

Conclusion

1	**Built**	A lightweight Linux task manager in C with a GTK 3 interface
2	**Monitored**	Reads /proc to show process, CPU and memory information
3	**Controlled**	Lets users select a process and send it a signal
4	**Learned**	Applied PIDs, process states, signals and permissions in practice

Thank you

Questions and Answers